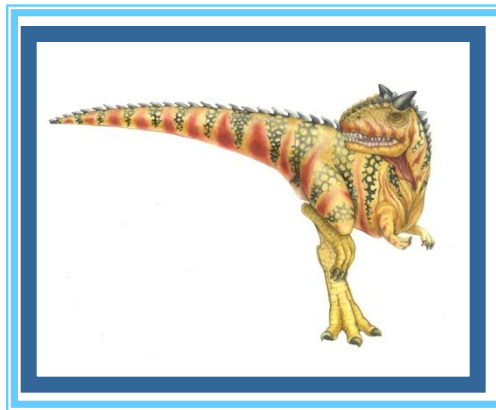


Chapter 11: File System Implementation





Chapter 11: File System Implementation

- File-System Structure
- File-System Implementation
- Directory Implementation
- Allocation Methods
- Free-Space Management





Objectives

- To describe the details of implementing local file systems and directory structures
- To describe the implementation of remote file systems
- To discuss block allocation and free-block algorithms and trade-offs





File-System Structure

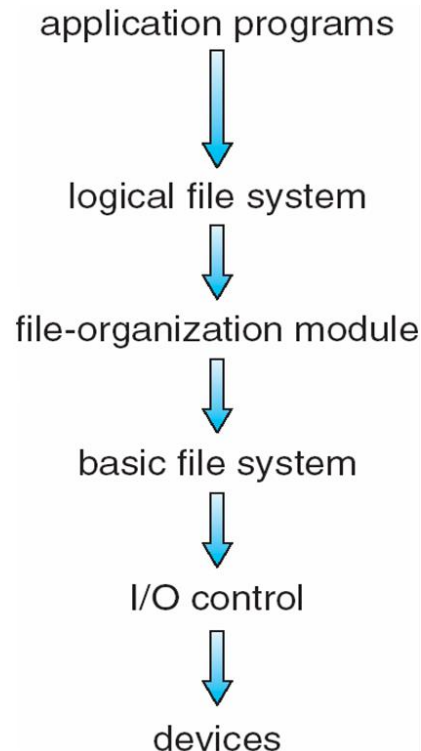
- File structure
 - Logical storage unit
 - Collection of related information
- File system resides on secondary storage (disks)
- File system organized into layers
- **File control block** – storage structure consisting of information about a file





Layered File System

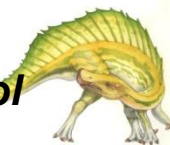
- Disks are usually accessed in physical blocks, rather than a byte at a time.
- Block sizes may range from 512 bytes to 4K or larger.
- File systems organize storage on disk drives, and can be viewed as a layered design:





Layered File System

- **physical devices:**
 - consisting of the magnetic media, motors & controls, and the electronics connected to them and controlling them..
- **I/O Control:**
 - Consists of device drivers and interrupt handlers.
 - Transfer information between main memory and the disk system.
- **Basic file system :**
 - Issue generic commands to the appropriate device driver to read and write physical blocs on the disk.
- **File organization module :**
 - knows about files and their logical blocks, and how they map to physical blocks on the disk.
 - In addition to translating from logical to physical blocks,
 - maintains the list of free blocks, and allocates free blocks to files as needed.
- **logical file system:**
 - deals with all of the meta data (i.e. everything about the file except the data itself).
 - manages the directory structure and the mapping of file names to **file control blocks, FCBs**





A Typical File Control Block

file permissions

file dates (create, access, write)

file owner, group, ACL

file size

file data blocks or pointers to file data blocks





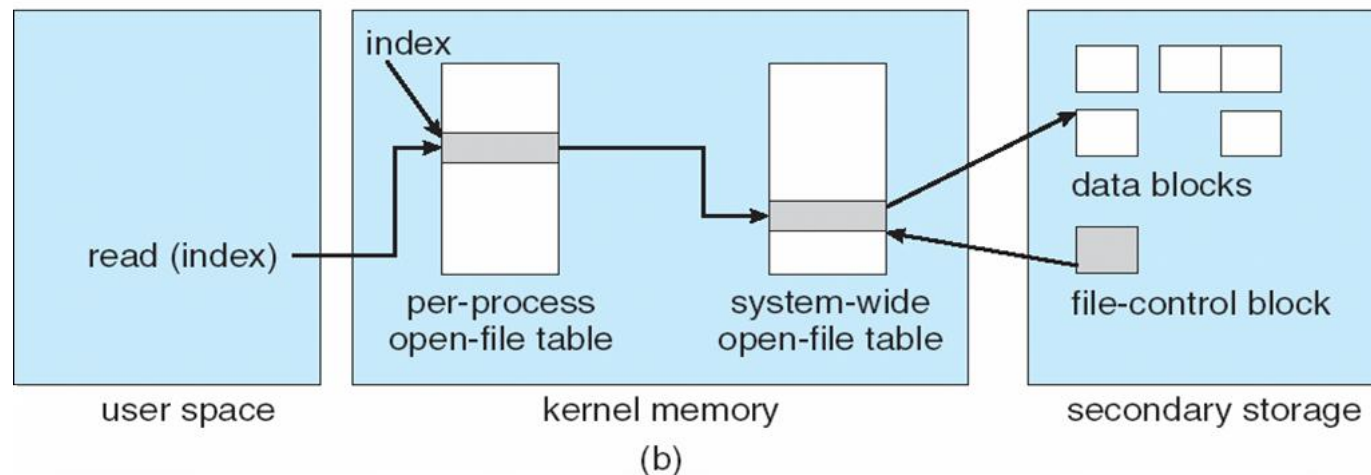
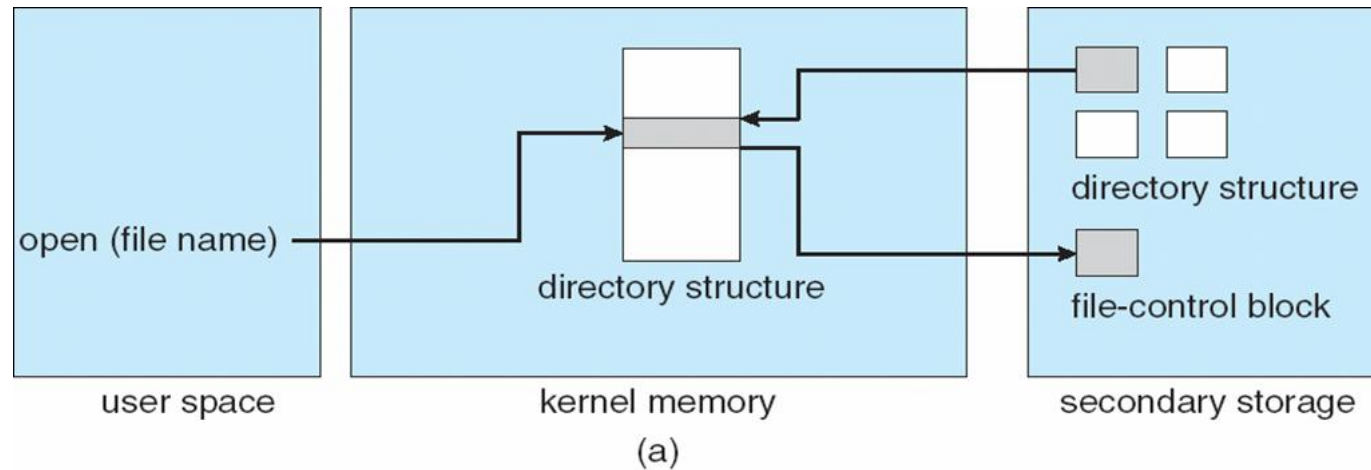
In-Memory File System Structures

- The following figure illustrates the necessary file system structures provided by the operating systems.
- Figure 12-3(a) refers to opening a file.
- Figure 12-3(b) refers to reading a file.





In-Memory File System Structures





Allocation Methods

- An allocation method refers to how disk blocks are allocated for files:
 - **Contiguous allocation**
 - **Linked allocation**
 - **Indexed allocation**





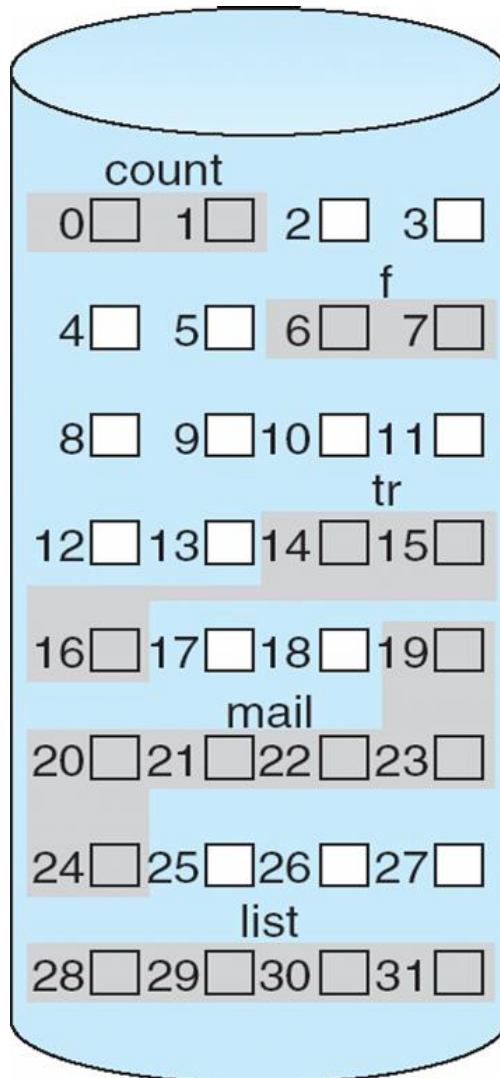
Contiguous Allocation

- Each file occupies a set of contiguous blocks on the disk
- Simple – only **starting location** (block #) and **length** (number of blocks) are required
 - Random access
 - Wasteful of space (dynamic storage-allocation problem)
 - ▶ External fragmentation problem
 - Files cannot grow





Contiguous Allocation of Disk Space



directory

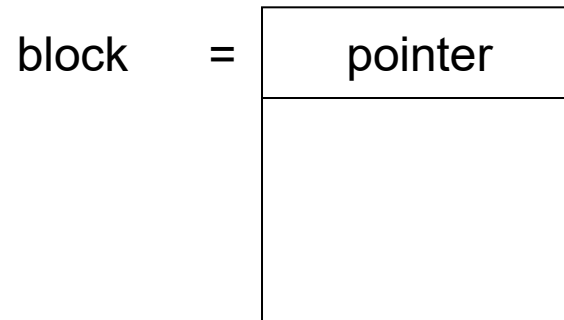
file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2





Linked Allocation

- Each file is a linked list of disk blocks: blocks may be scattered anywhere on the disk.





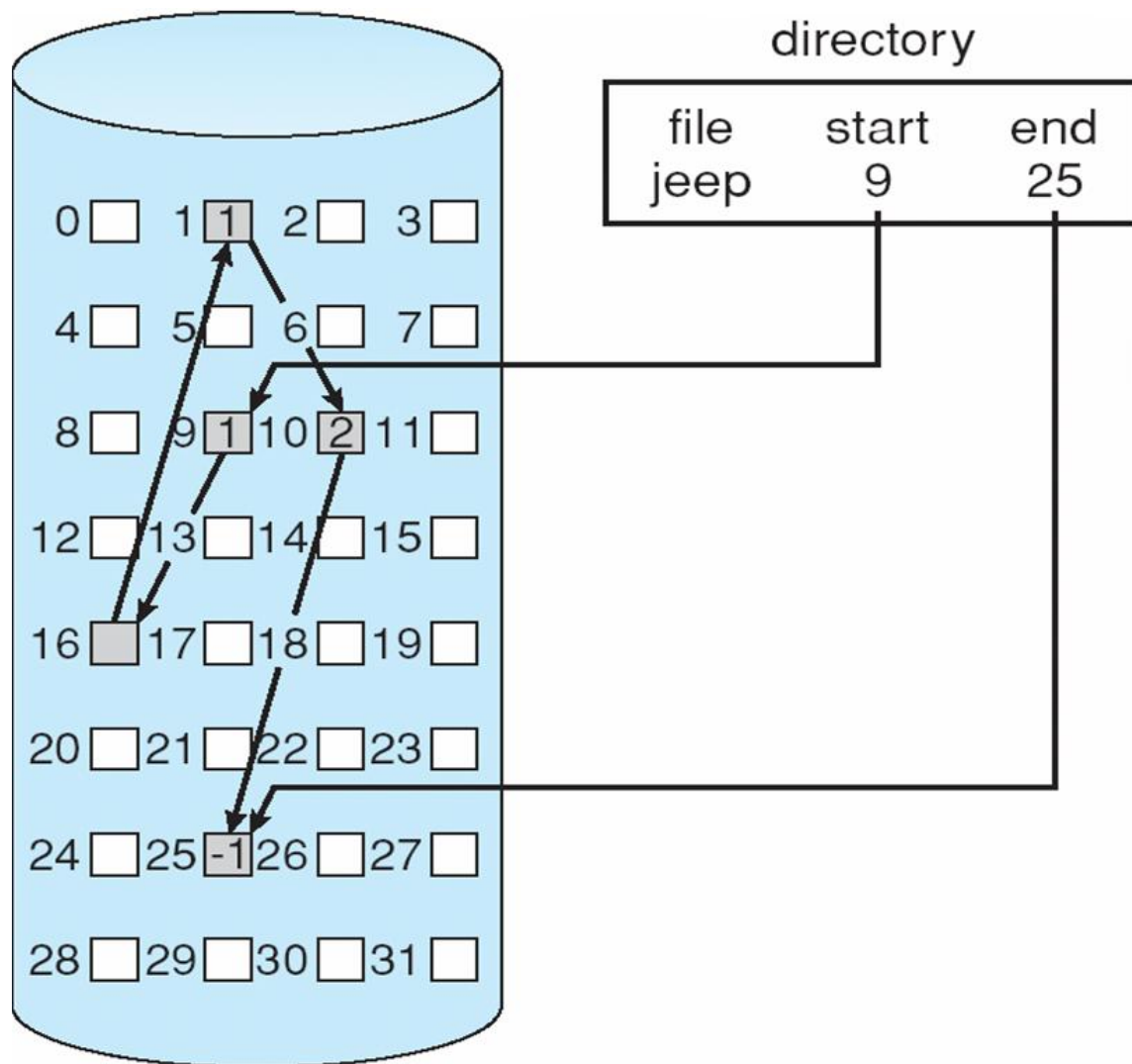
Linked Allocation (Cont.)

- Simple – need only starting address
- Free-space management system – no waste of space
- **No random access**
 - Good for sequential access but not efficient for direct access
- **File-allocation table (FAT)** – disk-space allocation used by MS-DOS and OS/2.





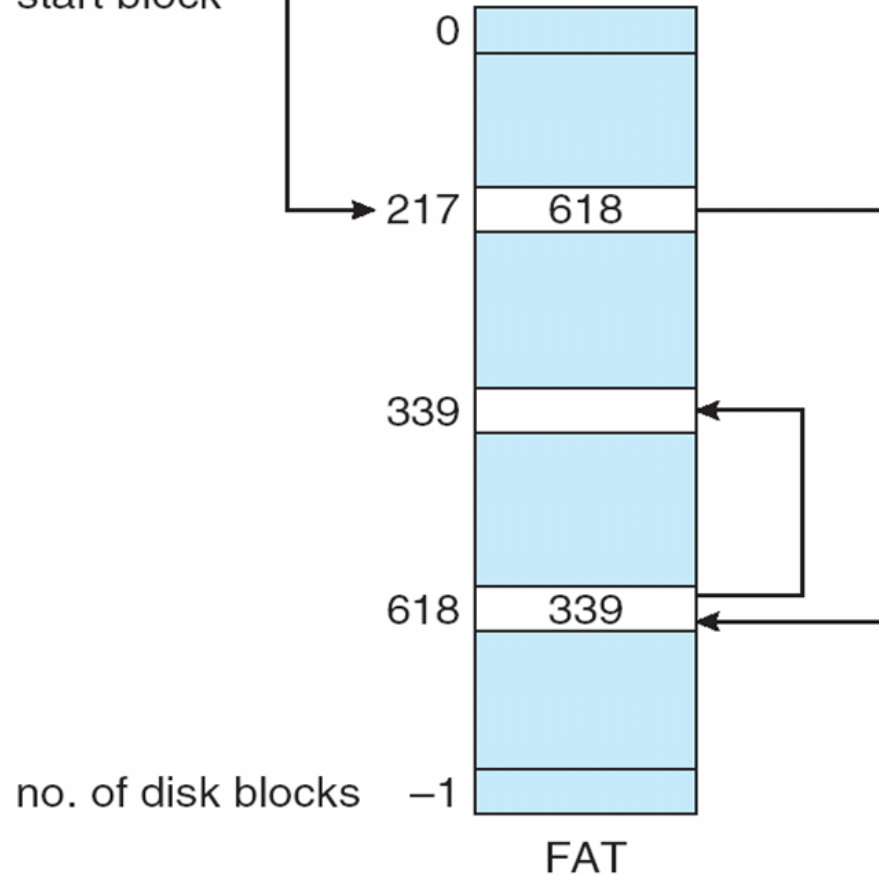
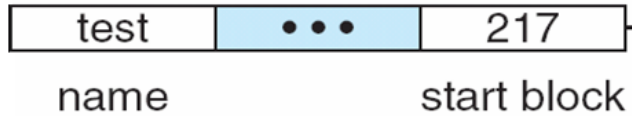
Linked Allocation





File-Allocation Table

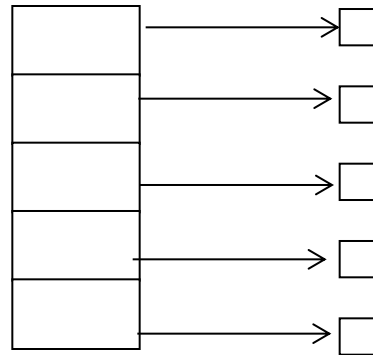
directory entry





Indexed Allocation

- Brings all pointers together into the ***index block***.
- Logical view.

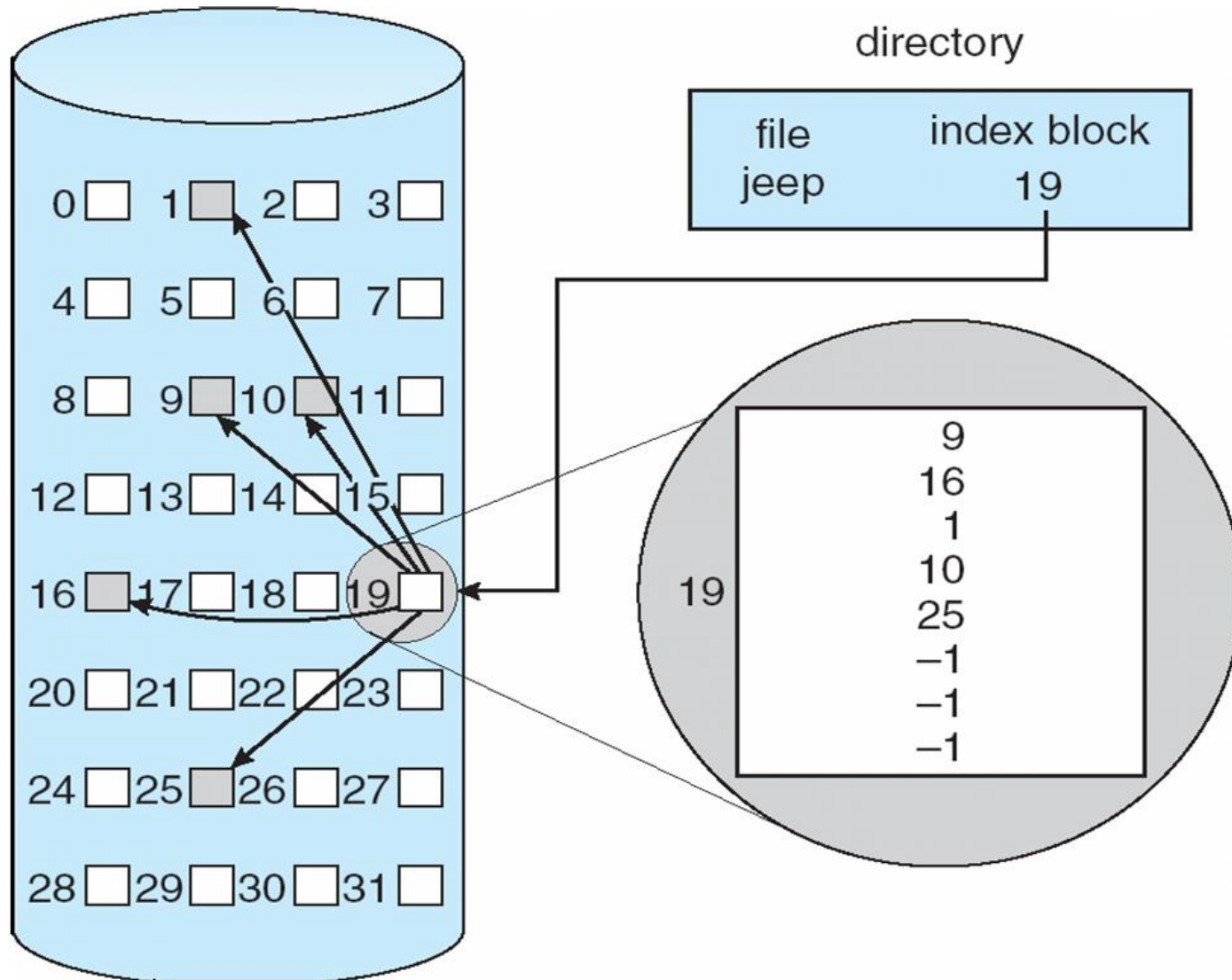


index table





Example of Indexed Allocation





Indexed Allocation (Cont.)

- ❑ Need index table
- ❑ Random access
- ❑ Dynamic access without external fragmentation, but have pointer **overhead of index block.**
- ❑ Mapping from logical to physical in a file of maximum size of 256K words and block size of 512 words. We need 1 block for index table.





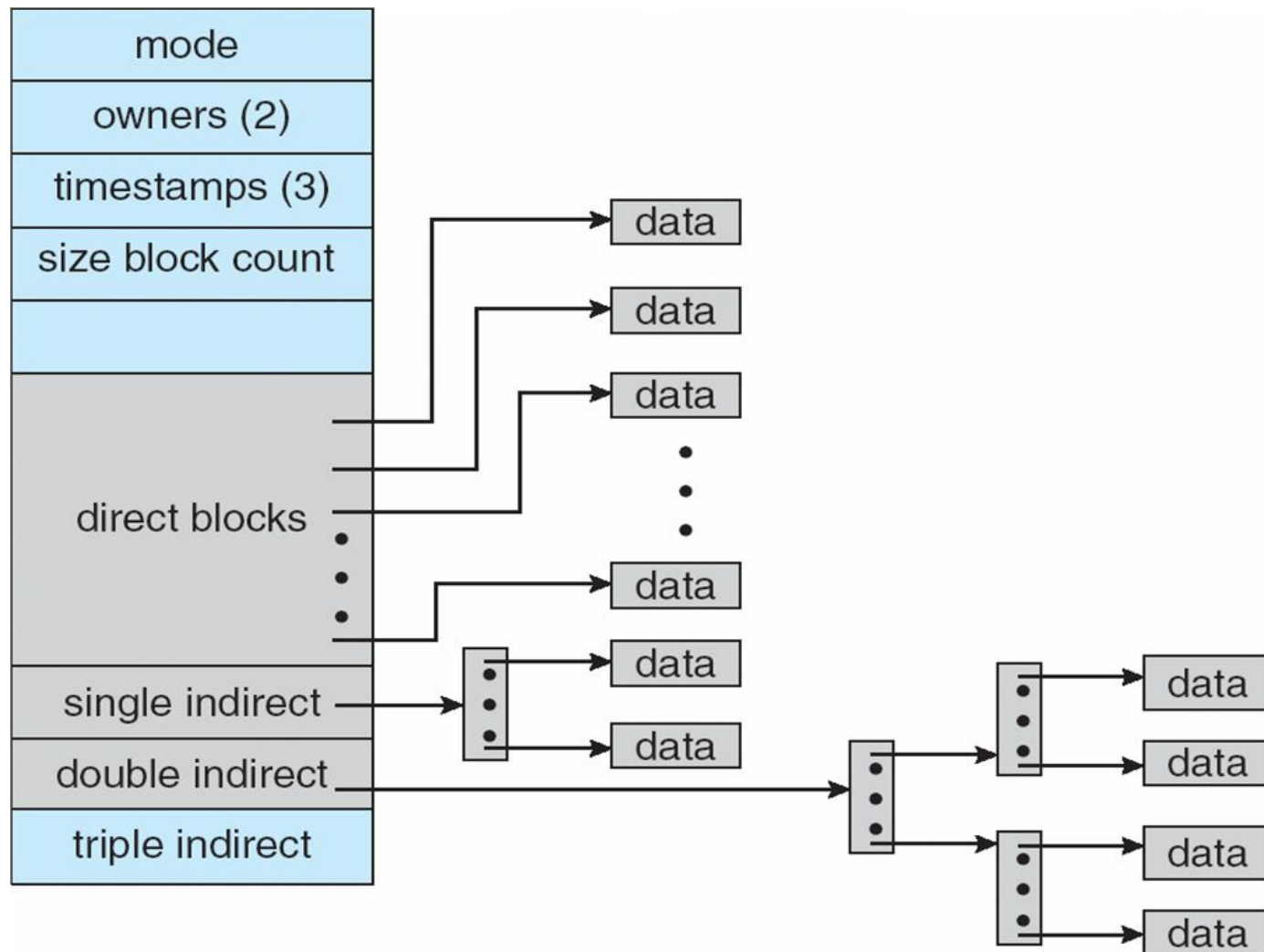
Indexed Allocation – Mapping (Cont.)

- **Linked scheme** – Link blocks of index table (no limit on size).
- **Multilevel index**-First level index block points to a second level index block, which further points to file blocks.
- **Combined Scheme**





Combined Scheme: UNIX (4K bytes per block)



The UNIX inode





Performance of Allocation Methods

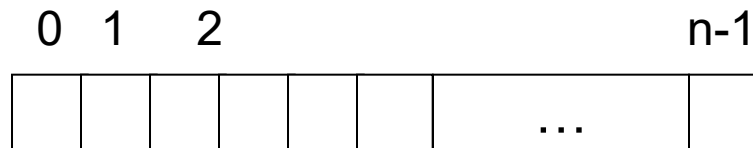
- ❖ The **optimal allocation method** is different for sequential access files than for random access files, and is also different for small files than for large files.
- ❖ Some systems support **more than one allocation method**, which may require specifying how the file is to be used (sequential or random access) at the time it is allocated. Such systems also provide conversion utilities.
- ❖ Some systems have been known to **use contiguous access for small files**, and **automatically switch to an indexed scheme when file sizes surpass a certain threshold**.
- ❖ And of course some systems adjust their allocation schemes (e.g. block sizes) to best match the characteristics of the hardware for optimum performance



Free-Space Management

Another important aspect of disk management is keeping track of and allocating free space.

Bit vector (n blocks)



$$\text{bit}[i] = \begin{cases} 0 \Rightarrow \text{block}[i] \text{ free} \\ 1 \Rightarrow \text{block}[i] \text{ occupied} \end{cases}$$

❑ For Example:

Consider a disk block where blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26 and 27 are free and the rest of the blocks are allocated.

The free space bit-map would be:

001111001111110001100000011100000 ...





Free-Space Management (Cont.)

- Bit map requires extra space

- Example:

block size = 2^{12} bytes

disk size = 2^{30} bytes (1 gigabyte)

$n = 2^{30}/2^{12} = 2^{18}$ bits (or 32K bytes)

- Easy to get contiguous files
- **Linked list (free list)**
 - Cannot get contiguous space easily
 - No waste of space
- **Grouping**
- **Counting**

